

Docker

From first principles to a runnable full-stack application

Dasari Sindhu

Software Engineer · AI/ML · Full-Stack
Bengaluru

LinkedIn: linkedin.com/in/dasari-sindhu

GitHub: github.com/sindhu0016

Contents

The chapters follow the order in which the ideas depend on each other. Each chapter starts on a new page.

- [1. About Docker](#)
- [2. Why Docker exists](#)
- [3. How Docker actually works](#)
- [4. Install and sanity-check](#)
- [5. Images and containers](#)
- [6. Your first containers](#)
- [7. Essential commands](#)
- [8. Writing a Dockerfile](#)
- [9. The .dockerignore file](#)
- [10. Layers, cache, and BuildKit](#)
- [11. Multi-stage builds](#)
- [12. Volumes and bind mounts](#)
- [13. Networking](#)
- [14. Environment, secrets, and config](#)
- [15. Docker Compose](#)
- [16. Health checks, restart, and logs](#)
- [17. Pushing to a registry](#)
- [18. Security basics](#)
- [19. Debugging when things break](#)
- [20. Mini project: FastAPI and React](#)
- [21. Command reference](#)
- [22. Interview questions](#)
- [23. What to learn next](#)

CHAPTER 1

About Docker

Definition. Docker is a platform for packaging an application, together with the runtime and libraries it needs, into a standard unit called a container. The same package then runs on a laptop, in continuous integration, or on a server, as long as a Docker engine is present.

A container is not a copy of an entire computer. It is a process, plus a private view of the filesystem, network, and process list. The host operating system kernel is shared. That is why a container starts in seconds and uses far less memory than a virtual machine, which must boot a second operating system.

Docker was introduced in 2013 by Solomon Hykes and Docker, Inc. Linux already had the isolation features. Namespaces keep one container from seeing another container's processes and network. Control groups, usually called cgroups, limit how much CPU and memory a container may use. Docker made those features usable through a command-line tool, an image format, and a registry. On macOS and Windows, Docker Desktop runs a small Linux virtual machine, and the engine lives inside it. The commands you type stay the same.

What Docker is, and what it is not

Docker is	Docker is not
A way to package an application with its runtime so the result is repeatable.	A virtual machine. Containers share the host kernel.
The base skill for Compose, and later for Kubernetes.	Kubernetes. Kubernetes schedules many containers across many machines. Learn Docker first.
A tool for local development, build artifacts, and production images.	A repair for incorrect code. A broken application inside a container is still broken.

The pieces you will meet

Piece	What it is	Chapter
Docker Engine	The background service, <code>dockerd</code> , that builds images and runs containers.	3
Docker CLI	The <code>docker</code> commands you type. The CLI talks to the engine.	7
Image	A read-only template: files, a default command, and metadata.	5
Container	A running or stopped instance created from an image.	5, 6
Dockerfile	The text recipe that builds an image.	8
Registry	A store for images, such as Docker Hub or GitHub Container Registry.	17
Compose	A YAML file that starts several containers as one application.	15

Official references

These notes are a study guide. The documents below are the canonical references when a flag or a file key is in doubt.

Document	What it covers
docs.docker.com	Platform overview and guides
docs.docker.com/get-docker	Install Docker Desktop or Engine
docs.docker.com/engine	Daemon, images, containers, storage, and networks
docs.docker.com/reference/dockerfile	Every Dockerfile instruction
docs.docker.com/build/building/best-practices	Layer cache, small images, non-root user
docs.docker.com/compose/compose-file	Compose file keys
docs.docker.com/reference/cli/docker	Every command and flag
hub.docker.com	Public images, including official ones
opencontainers.org	The open image and runtime specifications Docker follows

CHAPTER 2

Why Docker exists

A program can run on one laptop and fail on another machine for reasons that have nothing to do with the program's logic. One machine has Python 3.12 and the other has 3.10. A system library is present in development and missing on the server. PostgreSQL was assumed to be running, and it is not. That mismatch is the "it works on my machine" problem.

Docker answers it by shipping the application and its runtime together. The image records which Python, which libraries, and which start command were tested. Anywhere the engine can run that image, the process starts from the same filesystem.

Where Docker earns its place

- A new teammate can start the API, the website, and the database with one command.
- The artifact that passed tests is the artifact that is deployed. There is no second, hand-built server setup.
- Services stop fighting over the same host packages and, with a little care, over the same ports.
- A demo can be given to someone else without a long installation document.

Where Docker does not help

- It does not replace a virtual machine when you need a different kernel or a stronger isolation boundary.
- It does not schedule a fleet of machines. That is an orchestrator's job.
- It does not fix an application that binds the wrong address, reads the wrong configuration, or fails its own tests.

CHAPTER 3

How Docker actually works

You rarely talk to a container directly. You talk to a client. The client talks to the engine. The engine builds images, creates containers, and pulls or pushes registries.

```
graph TD
    You["You (CLI, Compose, or an IDE)"] --> Engine["Docker Engine (dockerd)"]
    Engine --> Images["+-- builds --> Images (read-only templates)"]
    Engine --> Containers["+-- runs --> Containers (processes with a writable layer)"]
    Engine --> Registry["Registry (Docker Hub, GHCR, or a private registry)"]
```

Piece	Role in the picture
CLI	Sends your command to the engine. It does not run the container itself.
Engine	The long-running service that does the work.
Image	The recipe's result. It does not execute until a container is created from it.
Container	A process plus an isolated view of files, network, and other processes.
Registry	Where images are stored so another machine can pull them.

What the kernel is doing

On Linux, two kernel features do most of the isolation. A **namespace** gives the container its own process list, hostname, mount points, and network interfaces. A process inside the container can believe it is the only process, even though the host kernel is shared. A **cgroup** caps CPU, memory, and sometimes disk. If you set a memory limit, the kernel enforces it.

On macOS and Windows, those Linux features are not the host kernel's native model. Docker Desktop starts a small Linux virtual machine and runs the engine there. Port mapping still looks like `localhost` on your machine, because Desktop forwards the ports. The commands in these notes do not change.

CHAPTER 4

Install and sanity-check

On macOS or Windows, install Docker Desktop. On Linux, install Docker Engine from the distribution, or Docker Desktop for Linux if you want the same application. After installation, the engine must be running before any command will succeed. A stopped whale icon, or a daemon that is not started, produces a connection error. That error means the client cannot find the engine. It does not mean your Dockerfile is wrong.

```
docker version
docker info
docker run --rm hello-world
```

`docker version` prints a client section and a server section. Both must appear. If only the client appears, the engine is not reachable. `docker info` shows storage, the number of containers, and whether you are in a Linux VM. `hello-world` pulls a tiny image, starts a container, prints a short message, and exits. The `--rm` flag deletes that container when it exits, so it does not linger in `docker ps -a`.

Compose v2. The command is `docker compose version`, with a space. The older `docker-compose` program, with a hyphen, is legacy. New projects, documentation, and interviews expect the spaced form. It is a plugin of the Docker CLI, not a separate product.

CHAPTER 5

Images and containers

This distinction unlocks the rest of the subject. An image is a template. It sits on disk and does not run. A container is created from an image and is a running process, or a process that has stopped and can be started again. One image can produce many containers, the way one class can produce many objects. Changing a container does not rewrite the image. The next container from that image starts clean, unless you mount storage that lives outside the container.

```
python:3.12-slim      image
|-- container A      uvicorn listening on port 8000
|-- container B      a one-off test command
```

Images are stored as layers. Each instruction in a Dockerfile usually adds a layer. Layers are cached and can be shared between images. That is why a rebuild is fast when only the last instruction changed, and slow when an early instruction changed. Chapter 10 explains how to order instructions so the cache survives ordinary code edits.

Names and tags

An image name has a repository and a tag: `name:tag`. The tag is a label, not a guarantee of freshness.

```
python:3.12-slim
ghcr.io/sindhu0016/gatepath-api:1.0.0
```

If you omit the tag, Docker uses `latest`. That word is only a default label. It is not “the newest image in the world,” and it moves whenever someone pushes again. In production, pin a version you have tested, or pin the image digest (`image@sha256:...`) when you need a bit-for-bit lock.

CHAPTER 6

Your first containers

This command downloads the Nginx Alpine image if it is not already local, creates a container named `web`, and maps port 8080 on your machine to port 80 inside the container. Nginx listens on 80 inside its own network namespace. Your browser cannot see that namespace directly, so the `-p` flag publishes it.

```
docker run --name web -p 8080:80 nginx:alpine
# open http://localhost:8080
# Ctrl+C stops the container, because it is in the foreground
```

Flag	Meaning
<code>-d</code>	Detached. The container keeps running and your terminal returns.
<code>-p host:container</code>	Publish a container port onto the host. The right-hand number is the port inside the container.
<code>--name</code>	A stable name. Without it, Docker assigns a random one.
<code>--rm</code>	Delete the container when it exits. Use this for experiments.
<code>-e KEY=value</code>	Set one environment variable for this run.
<code>-v host:container</code>	Mount a volume or a folder. See Chapter 12.
<code>-it</code>	Interactive terminal. <code>-i</code> keeps standard input open. <code>-t</code> allocates a terminal.
<code>--network</code>	Attach the container to a named network. See Chapter 13.

A throwaway shell is the fastest way to see that you are inside a different filesystem:

```
docker run --rm -it ubuntu:24.04 bash
```

The prompt is a shell in the container. `exit` ends it, and `--rm` removes the container. Nothing you install in that shell survives, because the writable layer is deleted with the container. Persistent data needs a volume, which is a later chapter.

CHAPTER 7

Essential commands

Learn this set before the long reference. Almost every later command is a variation of build, run, inspect, or remove.

```
# images
docker images          # local images (docker image ls)
docker pull python:3.12-slim
docker rmi IMAGE       # delete an image (docker image rm)

# containers
docker ps              # running only
docker ps -a           # running and stopped
docker logs -f CONTAINER # follow stdout and stderr
docker exec -it CONTAINER sh # shell in a container that is already running
docker stop CONTAINER
docker start CONTAINER
docker rm CONTAINER
docker rm -f CONTAINER # stop, then remove

# inspect and disk
docker inspect CONTAINER
docker stats
docker system df        # disk used by images, containers, volumes
docker system prune     # delete unused data; read the prompt
```

run versus exec. `docker run` creates a new container from an image. `docker exec` enters a container that already exists and is running. If the container has exited, `exec` fails. Start it, or use `docker run` with a different command.

`docker stop` sends a polite stop signal and waits, then kills the process if it does not exit. `docker rm` only removes a stopped container, unless you pass `-f`. Removing a container deletes its writable layer. It does not delete the image, and it does not delete a named volume.

CHAPTER 8

Writing a Dockerfile

A Dockerfile is a recipe. Docker reads it from top to bottom and produces an image. The file is usually named Dockerfile with no extension, in the directory you will pass as the build context.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Instruction	When it runs	Role
FROM	Build	The starting image. Every Dockerfile begins here, except a scratch image used on purpose.
WORKDIR	Build, and it becomes the start directory at run	Sets the directory for later commands. Docker creates it if needed.
COPY	Build	Copies files from the build context into the image. Prefer this over ADD.
ADD	Build	Like COPY, and it can also fetch a URL or unpack an archive. That extra behaviour is usually a surprise. Use COPY.
RUN	Build	Runs a command while the image is being built. Package installs belong here.
ENV	Baked in, available at run	An environment variable stored in the image. Good for PYTHONUNBUFFERED=1. Not for passwords.
ARG	Build only	A variable you pass with docker build --build-arg. It is not automatically present when the container runs.
EXPOSE	Documentation	Records which port the process listens on. It does not publish the port. You still need -p or Compose ports.
CMD	Run	The default command. Easy to replace: docker run IMAGE echo hi.
ENTRYPOINT	Run	The executable. Extra arguments are appended. Use it when the image is a specific program.
USER	Run	Drops root. Real images should do this. See Chapter 18.

HEALTHCHECK	Run, repeatedly	A command Docker runs to ask whether the process is usable, not merely started.
-------------	-----------------	---

```
docker build -t myapi:dev .  
docker run --rm -p 8000:8000 myapi:dev
```

The dot is the build context: the set of files sent to the engine. `.dockerignore` decides which of those files are left out. The `-t` flag names the resulting image.

Exec form. Prefer the JSON form of CMD and ENTRYPOINT. The process then runs directly, so `docker stop` and `Ctrl+C` deliver the signal to your program. The shell form (CMD `uvicorn ...` without brackets) wraps the program in a shell, and the shell may swallow the signal. The container then takes longer to stop, or leaves a child process behind.

CHAPTER 9

The .dockerignore file

`.dockerignore` works like `.gitignore`, but for the build context. A file listed here is not sent to the engine, so `COPY . .` cannot place it in the image. That matters for size and for secrets.

```
.git
.gitignore
**/.venv
**/node_modules
**/__pycache__
**/*.pyc
.env
.env.*
README.md
frontend/dist
```

Without this file, `COPY . .` can copy a virtual environment, `node_modules`, the Git history, and a `.env` file full of passwords. The image becomes enormous, builds slow down, and anyone who obtains the image can read the secrets from a layer. Put secrets in the environment at run time, not in the build context.

Also ignore files that would confuse the build, such as a local `.venv` created on Windows or macOS. A virtual environment built on your laptop will not match the Linux image. Chapter 11 creates the virtual environment inside the builder stage instead.

CHAPTER 10

Layers, cache, and BuildKit

Each of these instructions typically creates a layer. Docker reuses a layer when the instruction and the files it depends on are unchanged.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```

Dependencies change rarely. Application code changes often. Copying `requirements.txt` and installing it *before* `COPY . .` means an ordinary code edit does not rerun `pip`. If you copy the whole project first, every edit invalidates the install layer, and every build downloads packages again.

Combine related shell commands in one `RUN`, and clean up in that same instruction. A later `RUN rm` does not shrink the earlier layer. The deleted files remain in the history.

```
RUN apt-get update \
  && apt-get install -y --no-install-recommends curl \
  && rm -rf /var/lib/apt/lists/*
```

BuildKit is the modern builder and the default in current Docker. It caches more precisely, can run independent stages in parallel, and understands `--target` so you can build one stage of a multi-stage file.

```
docker build -t myapi:dev .
docker build --no-cache -t myapi:dev .
docker build --target runtime -t myapi:dev .
```

`--no-cache` is for when you suspect a stale layer, not for every build. Using it always throws away the speed layers are there to provide.

CHAPTER 11

Multi-stage builds

A production image should contain the program and what it needs to run. It should not contain compilers, package caches, or test tools. A multi-stage Dockerfile uses one stage to build and a later stage to run. Only what you explicitly copy moves forward. The builder stage is discarded from the final image.

A React build served by Nginx

Node compiles the frontend. Nginx serves the static files. The running container has no Node and no `npm`.

```
# syntax=docker/dockerfile:1
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:1.27-alpine AS runtime
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

`npm ci` installs exactly the lockfile. Copy the lockfile before the rest of the source so dependency installs stay cached. The Nginx configuration below sends `/api` to the Compose service named `api`, so the browser talks to one origin and Nginx talks to the API on the Docker network.

```
server {
    listen 80;
    root /usr/share/nginx/html;
    location / {
        try_files $uri $uri/ /index.html;
    }
    location /api/ {
        proxy_pass http://api:8000/;
    }
}

docker build -t gatepath-web:1.0.0 --target runtime ./frontend
```

A Python build that copies a virtual environment

Compilers needed to build some wheels stay in the builder. The runtime image receives the virtual environment and the application code, then drops root.

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim AS builder
WORKDIR /app
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim AS runtime
WORKDIR /app
RUN useradd --create-home --uid 10001 appuser
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
```

```
COPY --chown=appuser:appuser . .  
USER appuser  
EXPOSE 8000  
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Keep the requirements copy before the application source so the `pip` layer stays cached. Do not copy a virtual environment from your laptop. The builder creates a Linux environment that matches the image. A single-stage Node image is often around a gigabyte. The Nginx runtime is tens of megabytes. Smaller images pull faster, scan more cleanly, and contain fewer packages to patch.

CHAPTER 12

Volumes and bind mounts

A container has a thin writable layer on top of the image. Delete the container and that layer is gone. Databases, uploads, and anything else you must keep have to live outside that layer, in a volume or a bind mount.

Kind	What it is	Use it for
Named volume	A directory Docker manages, addressed by a name you choose.	Database files and persistent uploads.
Bind mount	A folder on your machine, mounted at a path inside the container.	Live source code during development.
Anonymous volume	Docker-managed storage with no name you chose.	Avoid it. It is easy to lose and hard to inspect.

A named volume for PostgreSQL

```
docker volume create pgdata
docker volume ls
docker volume inspect pgdata

docker run -d --name db \
  -e POSTGRES_PASSWORD=devpass \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16-alpine
```

The data remains after `docker rm db`, because it lives in `pgdata`, not in the container. `docker volume rm pgdata` deletes it. In Compose, declare `pgdata:` at the top level and mount `pgdata:/var/lib/postgresql/data` on the service. `docker compose down` keeps named volumes. `docker compose down -v` deletes them.

A bind mount for live code

```
docker run --rm -p 8000:8000 \
  -v "$(pwd)/backend:/app" \
  -w /app \
  myapi:dev \
  uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

The image still provides Python and the installed dependencies. The bind mount hides `/app` with your working tree, so the editor's saves are visible immediately. `--reload` restarts the process when those files change. This pattern is for development. A production container should run the code that was copied into the image, so the running bits match the built bits.

Do not bind-mount a production database from a laptop. A bind-mounted PostgreSQL directory from macOS or Windows is slow and can be damaged by an abrupt stop. A laptop disk is not a backup. Production data belongs in a named volume on the server, or in a managed database. Do not publish the data directory “so you can see the files.” Inspect with `docker exec` and take backups with `pg_dump`.

CHAPTER 13

Networking

On the default bridge network, containers do not get reliable DNS names for each other. Create a user-defined bridge and Docker's embedded DNS resolves container names, and Compose service names, to the right addresses.

```
docker network create appnet

docker run -d --name db --network appnet \
  -e POSTGRES_PASSWORD=devpass \
  -e POSTGRES_DB=gatepath \
  postgres:16-alpine

docker run -d --name api --network appnet \
  -e DATABASE_URL=postgresql://postgres:devpass@db:5432/gatepath \
  -p 8000:8000 \
  myapi:dev
```

The API connects to the hostname `db` on port 5432. That port was not published to your laptop, and it does not need to be. Containers on the same user-defined network use the container port directly. `-p` exists for programs on the host: your browser, a database GUI, or a colleague's machine. Publishing every internal port is how databases end up exposed on the internet.

localhost means "this container." Inside the API container, `localhost:5432` is the API itself, not PostgreSQL. The database hostname is `db`. On your laptop, `localhost:8000` is the published API port. Mixing those two meanings is the most common Compose failure.

Driver	When to use it
bridge	The normal choice. Prefer a user-defined bridge for any multi-container application.
host	The container shares the host network. There is no port isolation and no <code>-p</code> . Rare. Sometimes used for a tool that must see host ports directly.
none	No network. Useful for a one-off job that must not talk to anything.

```
docker network ls
docker network inspect appnet
docker network rm appnet
```

You can remove a network only when no container is still attached to it.

CHAPTER 14

Environment, secrets, and config

Configuration that changes between a laptop and a server belongs in the environment, not in the image. The image should be the same artifact everywhere. The environment tells it which database to call.

```
docker run -e DATABASE_URL=postgresql://postgres:devpass@db:5432/gatepath myapi:dev
docker run --env-file ./secrets.env myapi:dev
```

`-e` sets one variable. `--env-file` reads `KEY=value` lines. The file stays on the host. It enters the image only if a Dockerfile also copies it.

Never copy `.env` into the image. If that file is in the build context and you `COPY . .`, the values become a layer. `docker history` and a later `printenv` can reveal them. List `.env` in `.dockerignore`. Pass values at run time.

	ARG	ENV
When it exists	Build time. Pass it with <code>--build-arg</code> .	Stored in the image and present when the container runs.
Good for	A version label or a feature flag of the build.	Non-secret settings such as <code>PYTHONUNBUFFERED=1</code> .
Bad for	Passwords. Build arguments can show up in image history.	Passwords. They are baked into every copy of the image.

In Compose, `env_file` points at a file on the machine running Compose. `environment` sets individual keys and overrides the file. `${POSTGRES_PASSWORD}` is substituted from the shell or from a `.env` file sitting next to `compose.yaml`. That file is for Compose's own substitution. It is not automatically visible inside the container unless you pass it. For production, prefer a secret store, or secret files mounted at run time. Do not commit real passwords.

CHAPTER 15

Docker Compose

Compose runs several containers as one application. You describe services in YAML. Compose creates a project network, starts the containers, and gives each service a DNS name equal to its service name. Use `docker compose` with a space.

The file below matches the mini project in Chapter 20: PostgreSQL 16, a FastAPI service, and a frontend served by Nginx.

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-devpass}
      POSTGRES_DB: gatepath
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d gatepath"]
      interval: 5s
      timeout: 5s
      retries: 10
    # No ports. The API reaches db:5432 on the Compose network.
    # Publish 5432 only if a local GUI needs it, and bind it to 127.0.0.1.

  api:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://postgres:${POSTGRES_PASSWORD:-devpass}@db:5432/gatepath
    depends_on:
      db:
        condition: service_healthy
    ports:
      - "8000:8000"

  web:
    build: ./frontend
    depends_on:
      - api
    ports:
      - "8080:80"

volumes:
  pgdata:
```

`depends_on` with `condition: service_healthy` waits until PostgreSQL accepts connections. Waiting only until the container has started is too early: the database process can still be initialising, and the API will crash on the first connection. The web service waits until the API container has started. It does not, in this file, wait for an API health check. Add one if the frontend fails when it races the API.

```
docker compose up --build    # build, start, and show logs
docker compose up -d         # detached
docker compose logs -f       # all services
docker compose logs -f api   # one service
docker compose ps
docker compose exec api sh
```

```
docker compose down          # stop and remove containers; keep volumes
docker compose down -v       # also delete named volumes, including Postgres data
```

From the API container, the hostname `db` works. From the web container, the hostname `api` works. You do not use container IDs. A file named `compose.yaml` in the current directory is picked up automatically. `docker-compose.yml` still works. New projects should use `compose.yaml`.

CHAPTER 16

Health checks, restart, and logs

A container can be “running” while the program inside is stuck or still starting. A health check is a command Docker repeats to decide whether the service is usable. Compose uses that result for `service_healthy`.

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD curl -fsS http://127.0.0.1:8000/health || exit 1
```

`start-period` is grace time while the process boots. Failures during that window do not count toward `retries`. The check calls `127.0.0.1` because it runs inside the container, asking the local process whether it is up. Python slim images do not include `curl`. Either install it with `--no-install-recommends`, or avoid the extra package:

```
HEALTHCHECK CMD python -c "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/health')"
```

Keep `/health` cheap. It should answer that the process is up, without depending on the database. A separate `/ready` route can check PostgreSQL. Use the cheap route for the Docker health check if you do not want the container marked unhealthy every time the database blips. Use the ready route when traffic should wait for a real dependency.

Restart policy	Behaviour
no	The default. When the process exits, it stays down.
on-failure	Restart only after a non-zero exit. You can cap the number of retries.
always	Restart after any exit, and when the engine starts. A clean exit still comes back.
unless-stopped	Like <code>always</code> , except a container you stopped yourself stays stopped across a reboot.

```
docker run -d --restart unless-stopped --name api myapi:1.0.0
```

In Compose, set `restart: unless-stopped` on services that should survive a crash. Leave one-shot jobs on `no`.

Logs

Write logs to standard output and standard error. Docker collects those streams. A log file inside the container disappears with the writable layer, and `docker logs` will not show it.

```
docker logs -f api
docker logs --tail 100 api
docker compose logs -f --tail=200
```

Uvicorn already prints to standard output. In production, send that stream to your log system. Do not treat `docker exec` and `cat` of a file as the logging design.

CHAPTER 17

Pushing to a registry

An image you built exists only on that machine until you push it. A registry stores images so continuous integration, a teammate, or a server can pull the same bits. Tag a version you can roll back to. `latest` moves, so yesterday's latest is not a stable name.

Docker Hub

Hub names look like `username/repository:tag`.

```
docker login
docker tag myapi:1.0.0 sindhu0016/gatepath-api:1.0.0
docker push sindhu0016/gatepath-api:1.0.0
docker pull sindhu0016/gatepath-api:1.0.0
```

GitHub Container Registry

```
echo "$GITHUB_TOKEN" | docker login ghcr.io -u Sindhu0016 --password-stdin
docker tag myapi:1.0.0 ghcr.io/sindhu0016/gatepath-api:1.0.0
docker push ghcr.io/sindhu0016/gatepath-api:1.0.0
docker pull ghcr.io/sindhu0016/gatepath-api:1.0.0
```

GHCR packages are often private. The token needs permission to read and write packages. Also push the Git commit so a bad release can be pinned back:

```
docker tag myapi:1.0.0 ghcr.io/sindhu0016/gatepath-api:sha-abc1234
docker push ghcr.io/sindhu0016/gatepath-api:sha-abc1234
```

A version tag is the everyday name. A digest (`image@sha256:...`) is the lock you use when two builds must be identical even if someone moves the tag.

CHAPTER 18

Security basics

A container is a process with isolation, not a magic boundary. Treat it as a program that should run with as little privilege as it needs.

Do not run as root

Many official images default to root. If that process is compromised, root inside the container is a poor place to start, and some mounts can map that user onto a powerful user on the host.

```
RUN useradd --create-home --uid 10001 appuser
USER appuser
```

Create the user, give it ownership of the files it must write, then switch with `USER`. Bind-mounted files may need the same user id on the host. Named volumes can be initialised by the image's user.

Small bases and pinned tags

- Prefer `python:3.12-slim`, `node:22-alpine`, and `nginx:1.27-alpine` over an untagged `python` or a full Ubuntu image.
- Pin a tag you have tested. `3.12-slim` is a decision. `latest` is not.
- Alpine is small and uses musl. Some Python wheels expect glibc. Debian slim is the calmer default for Python. Alpine is a good default for Nginx and for many Node builds.

```
docker scout cves myapi:1.0.0
```

Docker Scout reports known vulnerabilities in the packages you shipped. The fix is usually a newer base image or a newer library, followed by a rebuild. Trivy is a common scanner in continuous integration.

Secrets and published ports

Anything you `COPY` or pass as a build `ARG` can leak through image history. Pass secrets at run time. Do not publish PostgreSQL on a cloud virtual machine with `-p 5432:5432`. Leave it on the internal network. If a GUI on your laptop needs it, bind the port to loopback only:

```
ports:
- "127.0.0.1:5432:5432"
```

That listens on the host's loopback interface, not on every network interface.

Optional extra limits

```
services:
  api:
    read_only: true
    security_opt:
      - no-new-privileges:true
    tmpfs:
      - /tmp
```

`read_only` makes the container filesystem read-only. Add a temporary filesystem at `/tmp` if the program needs scratch space. `no-new-privileges` blocks a process from gaining privileges through `setuid`. These are additional controls. They do not replace a non-root user or the rule about secrets.

CHAPTER 19

Debugging when things break

Work down this list. Most reports that “Docker is broken” are one of these mismatches.

1. Is the container running?

```
docker ps -a
docker compose ps
```

Exited (1) means the process crashed. A status that keeps returning to “restarting” means a restart policy is looping a crash. Read the logs before you rebuild.

2. Read the logs

```
docker logs --tail 200 api
docker compose logs -f api
```

Python tracebacks and Nginx errors appear here when the program writes to standard output. If the logs are empty, the program is writing to a file inside the container instead.

3. Open a shell when the command exits immediately

```
docker run --rm -it --entrypoint sh myapi:dev
docker compose run --entrypoint sh api
```

You get a shell even if the normal command crashes. Check files, run `printenv`, and try the start command by hand. `docker exec` works only while the container is still running.

4. Check the bind address

Uvicorn must listen on `0.0.0.0` to accept connections from outside the container: your browser, Nginx, and other services. Binding `127.0.0.1` makes the application reachable only from inside that container. The symptom is a published port in `docker ps` and a browser that cannot connect.

5. Check the hostname

From the API, PostgreSQL is `db`, not `localhost`. A typo such as `postgres` when the service is named `db` produces a name-resolution error. Confirm with:

```
docker compose exec api getent hosts db
```

6. Check permissions and image size

If the process runs as user id 10001 and a bind-mounted folder is owned by you, writes fail with permission denied. Do not write caches into that mount, or align the user ids, or keep generated files in a named volume. If the image is suddenly huge, `.dockerignore` is probably missing and a virtual environment or `node_modules` was copied in. `docker history myapi:dev` shows which layer grew.

```
docker inspect api
docker compose config # the YAML after variable substitution
docker system df
```

`docker compose config` shows the file Compose will actually use. It catches a wrong `DATABASE_URL` before you blame the application.

CHAPTER 20

Mini project: FastAPI and React

This is a small layout you can run. The Compose file in Chapter 15 is the one for this tree.

```
gatepath/
├── compose.yaml
├── backend/
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── requirements.txt
│   └── app/
│       ├── main.py          # FastAPI: /health, and routes that use Postgres
└── frontend/
    ├── Dockerfile          # Node 22 build, Nginx 1.27 runtime
    ├── nginx.conf
    ├── package.json
    └── src/
```

Backend image

```
FROM python:3.12-slim
WORKDIR /app
RUN useradd --create-home --uid 10001 appuser
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY --chown=appuser:appuser . .
USER appuser
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

backend/.dockerignore should exclude .venv, __pycache__, .env, and tests you do not want in the image. A minimum requirements.txt is fastapi, uvicorn[standard], and psycpg[binary]. The application reads DATABASE_URL from the environment. It listens on 0.0.0.0.

For a smaller and cleaner image, replace this single stage with the multi-stage Python file in Chapter 11. The single stage above is easier to read the first time. The multi-stage file is the one to keep.

Frontend image

Use the multi-stage Dockerfile in Chapter 11. Nginx serves the built files and proxies /api to http://api:8000/.

Run and stop

```
docker compose up --build
```

- Website: http://localhost:8080
- API health, if you want to call it directly: http://localhost:8000/health
- PostgreSQL is not published. The API reaches it at the hostname db.

You may keep local overrides in backend/.env. Keep that file out of Git and out of the image. The Compose environment block already injects DATABASE_URL. Stop with docker compose down. Delete the database volume, when you mean to, with docker compose down -v.

CHAPTER 21

Command reference

```
# build, run, stop
docker build -t NAME:TAG .
docker run --rm -p 8000:8000 NAME:TAG
docker run -d --name api NAME:TAG
docker stop api && docker start api
docker rm -f api

# shell and logs
docker exec -it api sh
docker logs -f api

# Compose v2
docker compose up --build
docker compose up -d
docker compose logs -f
docker compose ps
docker compose exec api sh
docker compose down
docker compose down -v

# inventory
docker images
docker ps -a
docker volume ls
docker network ls
docker inspect api
docker compose config
docker system df

# cleanup is destructive; read the prompt
docker system prune
docker system prune -a --volumes
```

`docker system prune` removes stopped containers, unused networks, and dangling images. Adding `-a` removes images not used by a container. Adding `--volumes` removes unused volumes, including database data you may still want. Run `docker system df` first, and do not prune volumes unless you intend to delete that data.

CHAPTER 22

Interview questions

What is the difference between an image and a container?

An image is a read-only template: a filesystem, a default command, and metadata. A container is a running or stopped instance of that image, with a writable layer on top. One image can start many containers. Deleting a container does not delete the image.

How is a container different from a virtual machine?

A virtual machine virtualises hardware and boots a guest operating system. A container shares the host kernel and isolates a process with namespaces and cgroups. Containers start faster and use less memory. They are not the same isolation boundary as a virtual machine.

What does EXPOSE do?

It documents the port the process listens on. It does not publish that port on the host. Publishing is `-p` or `Compose ports`.

What is the difference between CMD and ENTRYPOINT?

`CMD` is the default command and is easy to replace. `ENTRYPOINT` is the executable; extra arguments are appended to it. A common pattern is an entrypoint binary with `CMD` supplying default flags. Use the JSON exec form so stop signals reach the process.

Why is an image huge?

Typical causes: no `.dockerignore`, so `.git`, `node_modules`, or `.venv` were copied in; a full base image; compilers left in the final stage; no multi-stage build; package-manager caches not removed in the same `RUN`.

How do containers find each other?

Place them on a user-defined bridge. `Compose` does this automatically. Use the service name as the hostname and use the container port. `-p` is for the host, not for traffic between containers.

When do you use a bind mount, and when a named volume?

A bind mount is a host path. It is right for live source code. A named volume is storage Docker manages. It is right for database files. Do not bind-mount a production database from a laptop.

The application works locally and fails in Docker. What do you check?

The process may be binding `127.0.0.1` instead of `0.0.0.0`. It may be calling `localhost` instead of the service name `db`. A dependency may be missing from the image. Environment variables may not have been passed at run time. A file that exists on the laptop may never have been copied. Read the logs, then `docker compose config`, then a shell in the image.

Why did the layer cache break?

An early `COPY . .` invalidates every later layer when any file changes. Copy the lockfile, install dependencies, and copy the source last. A changed `RUN` line, `--no-cache`, or a build argument used before a `RUN` also forces that step to run again.

CHAPTER 23

What to learn next

At this point you can containerise a FastAPI and React application, describe it in Compose, and push a tagged image. Continue in this order.

1. **Compose Watch.** `docker compose watch` syncs files and rebuilds on change, which is often cleaner than a pile of development bind mounts.
2. **A build pipeline.** On a Git tag, build the image, scan it, and push it to GHCR with both a version tag and a commit tag.
3. **Scanning in that pipeline.** Docker Scout or Trivy should fail the build on critical vulnerabilities you are not prepared to ship.
4. **Kubernetes, only after Docker is fluent.** Kubernetes schedules containers across machines. Pods, Services, and Deployments are easier once images, ports, health checks, and configuration already feel ordinary. If Compose is still unclear, Kubernetes will hide the same bugs behind more objects.

Study notes, September 2026. Dasari Sindhu. Compose v2: `docker compose`, with a space.